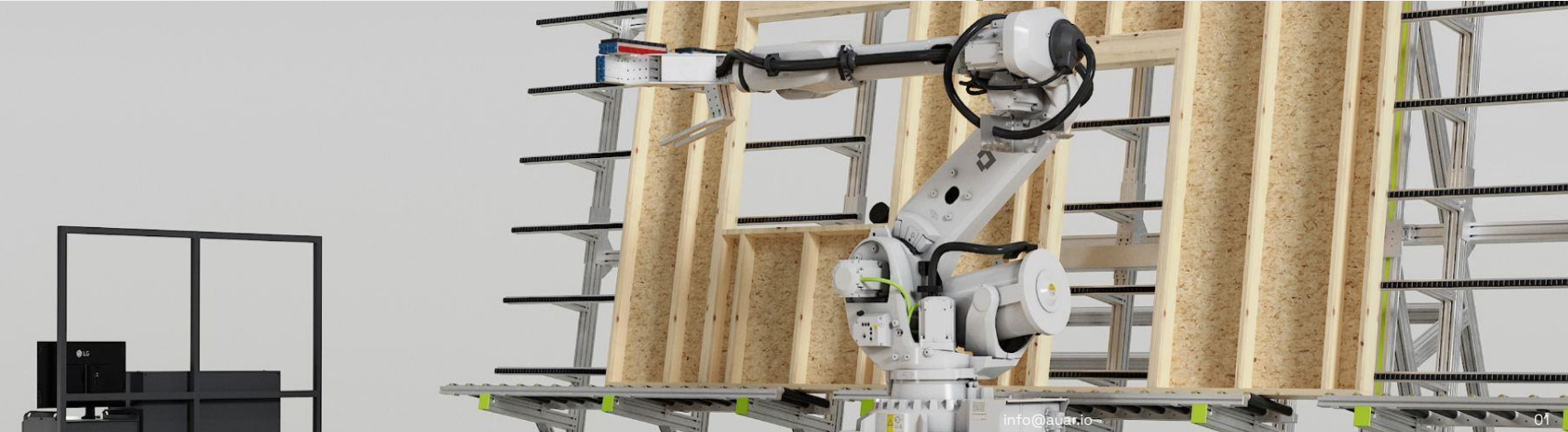


Technical Interview Task

Build the panel delivery system



Overview:

Technical Interview format for the AUAR Product Engineer Role

THE BRIEF

We give you a problem and some context. There is no spec to follow. We want you to decide what to build, what to prioritise, and what to leave out.

THE CONVERSATION

We will ask you to present your solution for 20 minutes, and explain every decision. We are more interested in your reasoning than your output; we ask *why* more than *what*.

THE TOOLKIT

Use AI tools, documentation, libraries, whatever you reach for day-to-day. We actively want to see that. We want to see your thinking and what you ship, not whether you can work from memory.



At a glance:

We want to see an end-to-end panel delivery planning system

Build a system that helps a **production manager** plan timber-panel deliveries and a **framer** execute them on site.

WHAT YOU'LL DO

01

Build

A working tool for both the production manager and the framer to plan and execute timber panel deliveries.
React + TS · Node + TS · one repo.

02

Design

A 20 minute presentation on users, scope, prioritisation, assumptions. *Format is up to you.*

03

Validation

Show us in your presentation how you would prove this works for real users, on a real build, on a real site.

USERS

Production manager · Framer

STACK

React · Node

DELIVERABLES

Repo · Presentation · Live demo

DEADLINE

The day before the interview



Context:

Panels, stacks, delivery.

AUAR's MicroFactory produces timber frame panels, the structural building blocks of a house. Each panel is manufactured to order and corresponds to a precise position in the building. A typical residential build requires 15–30 panels across two levels.

Once manufactured, panels are loaded onto delivery vehicles in stacks, and transported to the construction site. On site, the framer receives each delivery and assembles panels into the frame according to the build sequence. The order is determined by structural engineering requirements and cannot be changed.

A typical build spans 3–5 days. Deliveries are made on multiple days, and the sequence of deliveries matters as much as the sequence of panels within them. Currently the production manager plans deliveries on a spreadsheet, and the framer receives a WhatsApp message with a photo of a handwritten stack list.



15–30

PANELS PER BUILD

3–5

DAYS ON SITE

2

USERS TO SERVE



Users:

2 users. Both need to be served.

A planning tool the framer cannot use is not a complete solution.

Production Manager

CONTEXT

Works in the factory or site office. Plans which panels will go into each stack, in what order, and which stacks will go into each delivery. His plan informs the order in which the panels will be made by the MicroFactory. Coordinates between factory and construction site. Non-technical. Works on a desktop or tablet.

Framer

CONTEXT

Works on the construction site. Receives stack deliveries and unloads each stack to the most convenient location on the site, so that panels can be assembled into the frame in the correct sequence. Often working outdoors in varying conditions, wearing gloves, relying on a phone. Does not have a laptop.



Inputs:

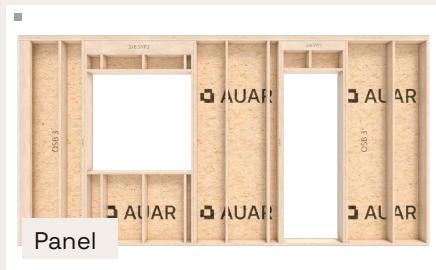
Example panel dataset

We give you an example set of panels that make up a build.
Take time to understand the data before building; the structure contains information you will need to make good decisions.

PRIMARY SOURCE

`interview-task-panels.json`

Treat the JSON as your primary source of structured information. Make reasonable assumptions about anything not captured in it.



Images are for domain context only.

Deliverables:

Three things to prepare

GitHub repository

Share the day before the interview. We review the code before we meet, so that we can come into the conversation with the most impactful questions for your interview!

20 minute presentation

Slides, diagrams, mind maps, sketches, or any combination, covering Part 2 & 3. You will walk us through it in the interview. Format is your choice; we care about the thinking, not the polish.

Live demo

Be ready to run the full stack and walk us through your decisions in the session, during your presentation.



PART 01 OF 03

Build

Design and build a system that helps the production manager plan deliveries and the framer execute them.



Requirements

- 01 React + TypeScript/Javascript on the frontend. Node.js + TypeScript/Javascript on the backend.
- 02 Keep both in the same repository.
- 03 Production-quality code, including types, error handling, structure.
- 04 Both users can do their job with what you build.
- 05 A small, well-designed thing is better than a large, half-finished one. We are looking for smart scope decisions, well written code, and evidence that you understood both users.



BONUS

AI integration: **only if** it
changes what a user can do.

If it genuinely changes what a user can do, add an AI capability.

Not for show and not because it is on-trend. Be prepared to explain the decision.



PART 02 OF 03

Design

Prepare a 20 minute presentation. Format is up to you: slides, diagrams, mind maps, sketches, a Figma file, photos of a whiteboard. A clear hand sketch beats a generic slide with bullet points.



Four questions to answer.

We care about how you think and communicate, not visual polish.

01

Your users

Who did you build for, and what does each user need? Walk through the specific decisions you made to serve those needs, including decisions about what not to build.

02

What you left out

What did you not build that you think matters most for this to work in the real world? Why did you leave it out?

03

The path to a finished product

If you were continuing this into a production-ready feature, what would you add and in what order? Walk us through your prioritisation: what comes first, what can wait, and why.

04

Assumptions

List the assumptions you had to make, e.g. about users, domain rules, data, or constraints. For each one, state: how would you go about discovering the real answer?



Validation

■ ————— ■

We want to understand how you would validate that this actually works for real users before trusting it on a live build. Share your thinking during your 20 minute presentation.



Three questions to answer.

Does this work for real users, on a real build, on a real site?

01

How would you test this with the actual users?

Before this goes live on a real project, how would you test it with a production manager and a framer? Walk through what that would look like: who you involve, what you ask them to do, and what you are watching for.

02

What does success look like after first real use?

Imagine this is used on a real build for the first time next week. What would tell you it worked, not technically, but practically? What would tell you it did not?

03

What could go wrong on site that your code cannot catch?

The failure modes that have nothing to do with bugs, i.e. the ways this breaks down in the real world, in the hands of real users, on a real construction site.



Four things we want you to know

ASSUMPTIONS

The brief is intentionally incomplete. You need to figure out what is missing, e.g. what the framer needs on site, what happens when a delivery does not go to plan, what the physical constraints of a delivery vehicle are. Make reasonable assumptions and document them.

SCOPE

Scope decisions matter as much as execution. Deciding what *not* to build, and being able to justify it, is a core part of this role.

WHY > WHAT

We will ask *why* more than *what*. In the interview, we are more interested in your reasoning than your output.

AI-NATIVE

Use whatever tools help you move fast. We want to see that you are AI-native, not that you can work without help. Just be ready to explain every decision.



The dataset, in summary.

20

Panels in this build

2

Levels (L1 + L2)

4

Panel types (ext / int / floor
/ roof)

PROJECT

Walthamstow Plot 3 — Unit A

Full dataset in `interview-task-panels.json`



Panel reference · max 3,050 × 6,096 mm



20 panels. 2 levels.

Day column shows the pre-planned delivery day where known — null indicates not yet scheduled. All dimensions in mm. Seq = install sequence (1 = first installed on site).

PANEL ID	TYPE	L	ZONE	WIDTH	HEIGHT	WT (KG)	SEQ	DAY	NOTES
EW-L1-N1	Ext. wall	1	north	3600	2700	148	1	1	Corner panel — must be plumb before EW-L1-N2 is set
EW-L1-N2	Ext. wall	1	north	2400	2700	98	2	1	Window opening — reduced weight
EW-L1-E1	Ext. wall	1	east	4800	2700	192	3	1	—
EW-L1-S1	Ext. wall	1	south	3600	2700	148	4	1	Corner panel — must be plumb before EW-L1-S2 is set
EW-L1-S2	Ext. wall	1	south	2400	2100	78	5	1	Front door opening — reduced height 2100mm
EW-L1-W1	Ext. wall	1	west	4800	2700	192	6	1	—
IW-L1-01	Int. wall	1	internal	3600	2700	85	7	1	Load bearing — structural sign-off required before install
IW-L1-02	Int. wall	1	internal	2400	2700	58	8	1	—
IW-L1-03	Int. wall	1	internal	1800	2700	44	9	1	Utility room partition
FC-01	Floor cass.	1	floor	1200	350	218	10	2	Spans east-west — do not deliver before all L1 external walls are installed and inspected
FC-02	Floor cass.	1	floor	1200	350	218	11	2	Spans east-west
FC-03	Floor cass.	1	floor	1200	350	218	12	2	Spans east-west
EW-L2-N1	Ext. wall	2	north	3600	2700	148	13	2	Corner panel
EW-L2-N2	Ext. wall	2	north	2400	2700	98	14	2	Bedroom window opening
EW-L2-E1	Ext. wall	2	east	4800	2700	192	15	2	—
EW-L2-S1	Ext. wall	2	south	3600	2700	148	16	2	Corner panel
EW-L2-W1	Ext. wall	2	west	4800	2700	192	17	—	Delivery day TBC — awaiting structural engineer sign-off on L1 floor
IW-L2-01	Int. wall	2	internal	3600	2700	85	18	—	Load bearing
IW-L2-02	Int. wall	2	internal	1800	2700	44	19	—	—
RF-01	Roof	2	roof	6000	200	235	20	—	Flat roof — mechanical lifting required. Must be last panel installed.

